

一、单项选择题

第 1~40 小题，每小题 2 分，共 80 分。下列每题给出的四个选项中，只有一个选项是最符合题目要求的。

23. 下面关于中断、异常和系统调用的叙述中，错误的是

- A. 中断或异常发生时，CPU 处于内核态
- B. 每个系统调用都有对应的内核服务例程
- C. 中断处理程序开始执行时，CPU 处于内核态
- D. 系统添加新类型设备时，需注册相应的中断服务例程

解答：本题选 A

A 错误。中断或异常发生时，CPU 处于用户态。当中断或异常发生时，CPU 会从用户态切换到内核态，然后开始执行相应的中断处理程序或异常处理程序。

B 正确。操作系统为每种系统调用提供了对应的内核服务例程，这些例程负责处理用户程序发起的系统调用，以及管理内核资源和执行相应的操作。当用户程序发起系统调用时，处理器会从用户态切换到内核态，并执行与该系统调用对应的内核服务例程。

C 正确。当中断发生时，CPU 会从用户态切换到内核态，并开始执行相应的中断处理程序。因为中断处理程序需要访问和操作受保护的内核资源，如管理设备、执行[特权指令](#)等，所以中断处理程序必须在内核态下执行。

D 正确。系统添加新类型设备时，需注册相应的中断服务例程。因为许多设备在发生特定事件时会触发中断，需要相应的中断处理程序来进行处理，该中断处理程序就是中断服务例程。

24. 下列选项中，操作系统在终止进程时不一定执行的是

- A. 终止子进程
- B. 回收分配的内存资源
- C. 撤销进程 PCB
- D. 回收进程占用的设备

解答：本题选 A

进程终止时，系统会回收分配给该进程的资源。当发生了终止事件后，操作系统便会调用终止原语，按下面步骤终止进程：

1. **检索 PCB 并获取其状态信息：**根据被终止进程的标识符，从 PCB 集合中检索出该进程的 PCB，并从该进程的 PCB 中读取该进程的状态。
2. **终止进程并更新调度标志：**若被终止的进程正处于执行状态，则立即终止该进程的执行，并置调度标志为真，以指示该进程被终止后应重新进行调度。
3. **终止子孙进程：**若该进程存在子孙进程，则还应终止其所有子孙进程，以防止它们成为不可控的进程。
4. **归还资源：**将被终止的进程所拥有的所有资源归还其父进程，或者归还给系统。
5. **将 PCB 从队列移出：**将被终止进程的 PCB 从所在队列（或链表）中移出，等待其它程序来搜集信息。

A 不一定执行。子进程是由一个现存的进程创建的新进程。在 Unix 和类 Unix 系统中，这是通过调用 `fork()` 系统调用来完成的。子进程将复制父进程的内存映像，包括代码段、数据段和[堆栈](#)。但注意，父进程和子进程并不共享内存，父进程可以通过 `wait()` 原语等待子

进程的执行完毕。当一个父进程终止时，它并不会自动终止其创建的子进程，子进程通常会继续运行。父进程可以在子进程执行的过程中被强制终止，但是这种情况可能会导致一些后续的问题，比如子进程成为**孤儿进程**（没有父进程的进程），并且子进程的结束状态可能需要由其他进程来处理。

B 一定执行。在终止进程时，操作系统必定会进行回收分配的内存资源，以防止内存泄漏和资源浪费。

C 一定执行。在终止进程时，操作系统需要撤销进程管理的 PCB (Process Control Block)，确保释放该进程的管理信息，包括状态、标识符、优先级等。

D 一定执行。在终止进程时，操作系统会回收进程占用的设备，以便其他进程可以使用。

25. 在支持页式存储管理的系统中，进程切换时操作系统要执行的操作是

- I. 更新程序计数器的值
 - II. 更新栈基址寄存器的值
 - III. 更新页基址寄存器的值
- A. 仅III
 - B. 仅 I、 II
 - C. 仅I、 III
 - D. I、 II、 III

解答：本题选 D

进程切换涉及以下操作：

- 1. **保存当前进程的上下文**：包括程序计数器，堆栈指针，寄存器等。
- 2. **恢复新进程的上下文**：加载新进程的程序计数器，堆栈指针，寄存器等。
- 3. **更新页表**：如果是页式存储管理系统，还需要更新页表等相关内容。
- 4. **权限检查**：确保新进程有足够的权限来运行。
- 5. **刷新 TLB 缓存**：如果存在 TLB，可能需要刷新以避免脏数据。

I 正确。程序计数器指示了将要执行的指令的地址，因此在进程切换时需要更新程序计数器以指向新进程的指令地址。

II 正确。栈基址寄存器用于指向栈帧的底部，通常用于保存函数调用的局部变量和参数。在进程切换时，当前进程的栈基址寄存器值需要被保存，而新进程的栈基址寄存器值则需要被加载，以确保新进程的栈环境正确。

III 正确。在页式存储管理中，不同的进程可能拥有不同的页表，因此在进程切换时，需要更新页基址寄存器的值，以便将内存访问转向新进程的页表。

综上所述，I、II、III正确。

26. 文件系统需占用部分外存空间记录空闲块的位置，占用外存空间大小与当前空闲块数量无关的是

- A. 位图法
- B. 空闲表法
- C. 成组链接法
- D. 空闲链表法

解答：本题选 A

本题考察文件存储空间的管理。

A 正确。位图法使用一个位图来表示磁盘的空闲块信息。位图中的每个位代表一个块的状态，1 表示已占用，0 表示空闲。位图占用外存空间大小与磁盘的总块数有关，与当前空闲块数量无关。

B 错误。空闲表法为每个文件分配一个连续的存储空间，也为外存上所有空闲区建立一张空闲表，每个空闲区对应一个空闲表项，其中包括表项序号、该空闲区的第一个盘块号、该区的空闲盘块数等信息。再将所有的空闲区按其起始盘块号递增的次序排列，形成空闲盘块表。若空闲块数量越多，则空闲表中的空闲表项越多，需要占用外存空间越大，因此空闲表法占用外存空间大小与当前空闲块数量有关。

C 错误。成组链接法通过将多个物理块组织成一个组，形成链接列表来管理存储空间。成组链接法是将空闲表法和空闲链表法结合而形成的一种空闲盘块管理方法。适用于大型文件系统。UNIX 系统中采用该方法进行文件存储空间的管理。当空闲块被分配或回收时，成组链接法需要更新这些链接，因此成组链接法占用外存空间大小与当前空闲块数量有关。

D 错误。空闲链表法是将所有空闲盘区拉成一条空闲链。根据构成链所用基本元素的不同，可把链表分成两种形式：空闲盘块链和空闲盘区链。空闲盘块链是将磁盘上的所有空闲空间以盘块为单位拉成一条链。每个盘块都有一个指向下一个盘块的指针。空闲盘区链是将磁盘上的所有空闲空间以盘区为单位拉成一条链。每个盘区包含若干个盘块，并有一个指向下一个盘区的指针。与空闲盘块链不同，每个盘区还记录了该盘区的大小信息。空闲链表的长度随着空闲块的分配和回收而变化，因此，空闲链表法占用外存空间大小与当前空闲块数量有关。

27. 下列算法中，每次回收分区时仅合并大小相等的空闲分区的是

- A. 伙伴算法
- B. 最佳适应算法
- C. 最坏适应算法
- D. 首次适应算法

解答：本题选 A

A 正确。伙伴算法即伙伴分配算法 (Buddy Allocation Algorithm) 将可用内存分割成大小相等、成对的“伙伴”块。当请求内存时，系统会找到大于等于请求大小的最小块，并分配给请求的大小，将剩余部分形成新的伙伴块。这种方法有利于减少外部碎片，但可能会生成一定量的内部碎片。

B 错误。最佳适应算法 (Best Fit Algorithm) 是指在内存分配时，选择能够恰好满足需求并且剩余空间最小的可用空闲分区进行分配。

C 错误。最坏适应算法 (Worst Fit Algorithm) 是指在内存分配时，选择能够满足需求的最大的可用空闲分区进行分配。

D 错误。首次适应算法 (First Fit Algorithm) 是指在内存分配时，选择第一个能满足要求的空闲分区进行分配。

28. 若进程 P 中线程 T 先打开文件，得到文件描述符 fd，再创建线程 Ta 和 Tb，则下列资

源中，线程 Ta 与 Tb 可共享的是

- I. 进程 P 的地址空间
 - II. 线程 T 的栈
 - III. 文件描述符 fd
- A. 仅 I
 - B. 仅 I、III
 - C. 仅 II、III
 - D. I、II、III

解答：本题选 B

本题考察进程和线程的关系。线程可共享其所属进程的地址空间和资源，包括堆和全局变量。每个线程都有自己的栈，用于保存其局部变量和返回地址。

I 正确。线程可共享其所属进程的地址空间。

II 错误。不同线程的栈是独立的，不可共享。

III 正确。线程可以共享其所属进程的打开的文件描述符。

综上所述，仅 I、III 正确。

。

29. 以下系统调用的实现中，包含文件按名查找功能的是

- A. open()
- B. read()
- C. write()
- D. close()

解答：本题选 A

文件按名查找功能指的是通过文件名来获取文件的操作，比如打开一个文件以便读取或写入数据。在 Unix 和类 Unix 系统中，这通常是通过 open() 系统调用来实现的。

A 正确。open() 系统调用用于打开文件，是通过文件按名查找实现的，open() 系统调用返回一个文件描述符，该描述符用于后续的文件操作，如读取、写入等。

B 错误。read() 系统调用用于从文件中读取数据。read() 需要再文件已被打开的情况下才能实现。

C 错误。write() 系统调用用于向文件中写入数据。write() 需要再文件已被打开的情况下才能实现。

D 错误。close() 系统调用用于关闭一个打开的文件。close() 需要再文件已被打开的情况下才能实现。

只有文件被打开后才能对该文件进行读、写、关闭等操作，所以 B、C、D 都是 A 的后续操作。所以四个选项中第一步是打开文件，只有正确查找文件才能正确打开文件，因此只有 open() 包含文件按名查找功能。

30. 假设某系统使用时间片轮转调度算法进行 CPU 调度，时间片大小为 5ms，系统共有 10 个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若**队尾的进程 P**所需 CPU 时间最短，时间为 25ms。在不考虑系统开销的情况下，则进程 P 的周转时间为

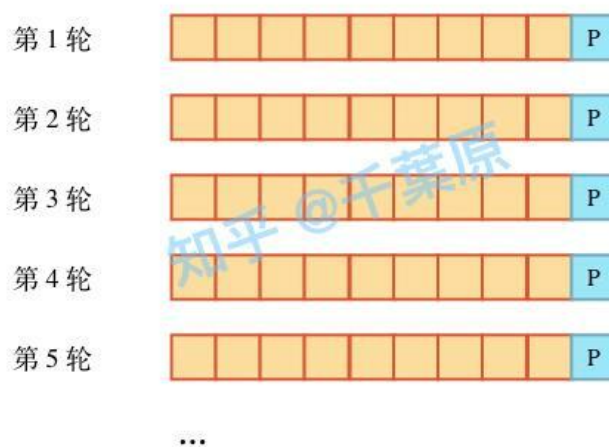
- A. 200 ms
- B. 205 ms
- C. 250 ms
- D. 295 ms

解答：本题选 C

在时间片轮转(Round Robin, RR)调度算法中，进程轮流执行一个固定的时间片。如果一个进程在时间片结束时还没有完成，它将被放到就绪队列的末尾，等待下一轮调度。

根据题目条件，时间片大小为 5 ms，进程 P 所需 CPU 时间为 25ms，需要 $25\text{ms} / 5\text{ms} = 5$ 个时间片才能完成其执行，即需要 5 轮才能完成。

又因为系统共有 10 个进程，初始时均处于就绪队列，每轮每个进程占用一个时间片，每轮总时间 $10 \times 5\text{ms} = 50\text{ms}$ ，因为进程 P 所需 CPU 时间最短，所以进程 P 第一个执行完毕，即在 P 执行完毕前，别的进程还有时间片待分配，又进程 P 位于队尾，所以每轮进程 P 最后一个执行，也就是 P 所在轮占用时间片结束时，这个轮次结束。根据之前的分析，该过程总共需要 5 轮，总计 $5 \times 50\text{ms} = 250\text{ms}$ 。该过程示意图如下。



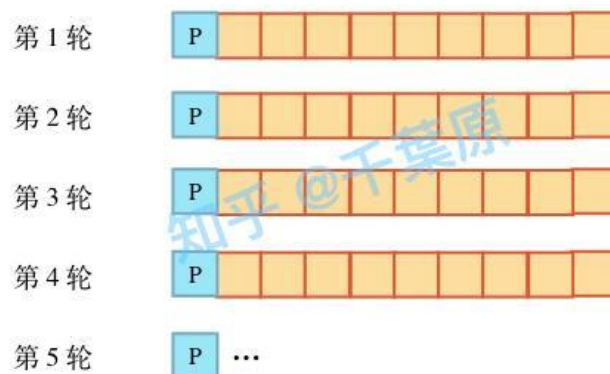
进程 P 的完成时间为 250ms，到达时时间为 0ms，因此进程 P 的周转时间 = 完成时间 - 到达时间 = $250\text{ms} - 0\text{ms} = 250\text{ms}$ 。

【举一反三】

假设某系统使用时间片轮转调度算法进行 CPU 调度，时间片大小为 5ms，系统共有 10 个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若队首的进程 P 所需 CPU 时间最短，时间为 25ms。在不考虑系统开销的情况下，则进程 P 的周转时间为

- A. 200 ms
- B. 205 ms
- C. 250 ms
- D. 295 ms

参考答案：B，示意图如下：

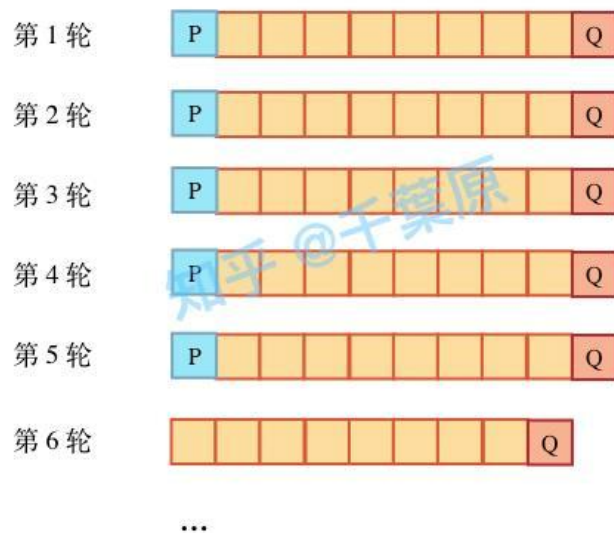


【举一反三】

假设某系统使用时间片轮转调度算法进行 CPU 调度，时间片大小为 5ms，系统共有 10 个进程，初始时均处于就绪队列，执行结束前仅处于执行态或就绪态。若队首的进程 P 所需 CPU 时间最短，时间为 25 ms，队尾的进程 Q 所需 CPU 时间第二短，时间为 30 ms，在不考虑系统开销的情况下，则进程 Q 的周转时间为

- A. 200 ms
- B. 205 ms
- C. 250 ms
- D. 295 ms

参考答案：D，示意图如下。



31. 键盘中断服务例程执行结束时，所输入的数据存放位置是

- A. 用户缓冲区
- B. CPU 中的通用寄存器
- C. 内核缓冲区
- D. 键盘控制器的数据寄存器

解答：本题选 C

当键盘输入数据时，首先会将数据发送到键盘控制器的数据寄存器中。当键盘控制器的数据寄存器接收到数据后，它会触发一个中断请求 (IRQ1)，通知 CPU 有数据等待处理。CPU 响应中断请求后，会执行键盘中断服务例程。在中断服务例程中，CPU 从键盘控制器的数据寄存器读取输入数据，并将其存放到内核缓冲区中。

A 错误。用户缓冲区通常指的是应用程序为接收输入数据而分配的缓冲区。在键盘中断服务例程执行结束时，输入数据并不会直接存放在用户缓冲区中，而是先存放在内核缓冲区中。应用程序可以通过系统调用或其他机制从内核缓冲区读取输入数据，并将其复制到用户缓冲区中。

B 错误。CPU 中的通用寄存器并不是输入数据的最终存放位置。在键盘中断服务例程执行过程中，输入数据可能会暂时存放在 CPU 的通用寄存器中，但最终会被存放到内核缓冲区中。

C 正确。内核缓冲区是操作系统用于暂存输入数据的内存区域。在键盘中断服务例程执行过程中，输入数据被读取并存入内核缓冲区，以便后续的系统调用或应用程序访问。内核缓冲区通常由操作系统管理，确保数据的正确性和一致性。

D 错误。键盘控制器的数据寄存器是输入数据的初始存放位置，但并不是中断服务例程执行结束时的最终存放位置。

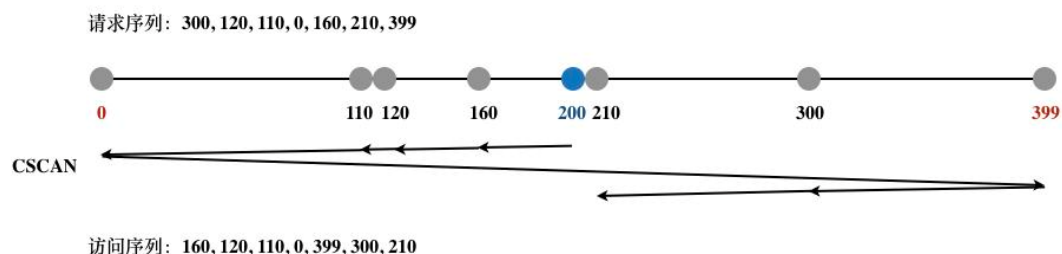
32. 某磁盘的磁道数为 400（磁道号为 0~399），采用循环扫描算法(CSCAN)进行磁盘调度，完成对 200 号磁道的请求后，磁头向磁道号减小的方向移动，若还有 7 个请求，对应的磁道号分别为 300, 120, 110, 0, 160, 210, 399，则完成上述磁盘请求后磁头移动的距离是

- A. 599
- B. 619
- C. 788
- D. 799

解答：本题选 C

循环扫描算法 (CSCAN)是扫描算法 (SCAN) 的改进，在磁头移动方向上选择与当前磁头所在磁道距离最近的请求作为下一次服务的对象，但与 SCAN 算法不同的是，当磁头到达磁盘的一端并完成所有请求后，它会立即返回到磁盘的另一端，而不是继续扫描直到没有请求为止。

题目要求从 200 号磁道向磁道号减小的方向移动，见如下示意图。



磁头移动的距离为 $|0-200|+|399-0|+|210-399|=788$

二、综合应用题

第 41~47 小题，共 70 分。

45.（本题 7 分）某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址和物理地址长度均为 32 位，页表项的大小为 4 字节，页大小为 4MB，虚拟地址结构如下。

页号（10 位）	页内偏移量（22 位）
----------	-------------

进程 P 的页表起始虚拟地址为 B8C0 0000 H，被装载到从物理地址 6540 0000 H 开始的连续主存空间中。

请回答下列问题，要求答案用十六进制表示。

(1)若 CPU 在执行进程 P 的过程中，访问虚拟地址 1234 5678 H 时发生了缺页异常，经过缺页异常处理和 MMU 地址转换后得到的物理地址是 BAB4 5678 H，在此次缺页异常处理过程中，需要为所缺页分配页框并更新相应的页表项，则该页表项的虚拟地址和物理地址分别是什么？该页表项的页框号更新后的值是什么？（3 分）【注：页框，frame】

(2)进程 P 的页表所在页的页号是多少？该页对应的页表项的虚拟地址是多少？该页表项中的页框号是多少？（4 分）

解答：

(1) 第一问【该页表项的虚拟地址和物理地址分别是什么】

进程页表的页表项虚拟地址（即进程页表的起始虚拟地址），是 B8C0 0120 H，物理地址是 65400120 H。

虚拟地址和物理地址长度均为 32 位，虚拟地址结构如下：

页号（10 位）	页内偏移量（22 位）
----------	-------------

【注：进程虚拟地址/逻辑地址从 0 开始】进程 P 的虚拟地址空间起始地址为：0000 0000 H

= 0000000000 000000000000000000000000 B，

页号 页内偏移

虚拟地址空间的起始地址所在页的页号为 0000000000 B= 0 H。

当进程 P 访问虚拟地址 1234 5678 H 时，该地址对应的页号、页内偏移分别为：

1234 5678 H

= 0001 0010 0011 0100 0101 0110 0111 1000 B，

页号 0001 0010 00 B= 048 H，进程 P 对虚拟地址 1234 5678 H 进行地址变换时，需要访问页表中的第 48 H 号表项。

已知页表项的大小为 4 字节，每项占 4 个地址单元。进程 P 的页表的起始虚拟地址为 B8C0 0000 H，即页表的第 0 号项存放在从虚拟地址 B8C0 0000 H 开始的连续 4 个字节中。页表的第 48 H 号项的虚拟地址为 $B8C0\ 0000\ H + 48\ H \times 4 = B8C0\ 0000\ H + 48\ H \ll 2 = B8C0\ 0120\ H$ 。

因为该页表被装载到的主存空间连续，页表起始虚拟地址为 B8C0 0000 H 对应的物理地址为 6540 0000 H，页表的 48 H 号项的物理地址为 $6540\ 0000\ H + 48\ H \times 4 = 6540\ 0000\ H + 48\ H \ll 2 = 6540\ 0120\ H$ 。

页表的中各个表项的虚拟地址、物理地址如下所示：

页表项号	页表项虚拟地址	页表项物理地址
0 H	B8C0 0000 H	6540 0000 H
1 H	B8C0 0004 H	6540 0004 H
...	...	...
48 H	B8C0 0120 H	6540 0120 H
...	...	...

第二问【该页表项的页框号更新后的值是什么】

此问考查进程页表中第 48H 号页表项的内容。

答案：该页表项的页框号【注：指物理地址中对应页框号的高 10 位】更新后的值是 2EA H。

第 48H 号页表项存储的是虚拟地址 1234 5678 H 的物理地址。根据题意，经过缺页异常处理和 MMU 地址转换后，该虚拟地址对应的物理地址是：

BAB4 5678 H

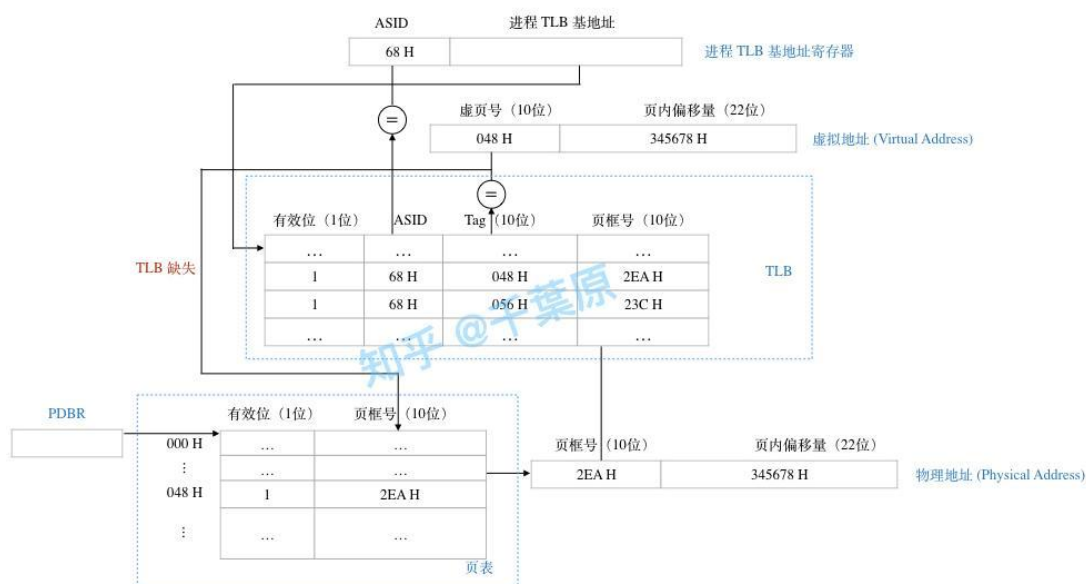
= 1011 1010 10 11 0100 0101 0110 0111 1000 B。

，其中的页框号为 10 1110 1010 B = 2EA H，即页号 048 H 映射到页框号 2EA H。

虚拟地址 1234 5678 H 和物理地址是 BAB4 5678 H 的页内/页框内偏移量都是

11 0100 0101 0110 0111 1000 B = 345678 H

假设页表带 TLB，更新后 TLB 和页表如下图所示。



(2) 【第 (2) 小题考察进程 P 的页表中页表项的地址，包括虚拟地址、物理地址。

前提假设：

进程 P 的页表本身也在进程 P 的地址空间中，因此页表本身也占据 1 个或多个 page，在页表中也有相应的表项。

如何理解：进程的页表属于 OS 内核数据结构，位于由操作系统管理的内核空间中。在

Unix/Linux 系统中，将内核空间映射到进程的虚拟地址空间（即用户空间）中，进程利用页表访问进程用户空间中的代码/数据，以及内核空间中的页表。

】

第一问【进程 P 的页表所在页的页号是多少】答案：进程 P 的页表所在页的页号是 2E3 H。

P 的页表的起始虚拟地址为：

B8C0 0000 H = 1011100011 000000000000000000000000 B，页号 10 1110 0011 B = 2E3 H。

第二问【该页对应的页表项的虚拟地址是多少】答案：该页对应的页表项的虚拟地址是 B8C0 0B8C H。

根据题意，已知页表起始虚拟地址为 B8C0 0000 H，即页表第 0 号项虚拟地址为 B8C0 0000 H，页表项的大小为 4 字节。因此，页表中第 2E3 H 号项的虚拟地址为：

$B8C0\ 0000\ H + 2E3\ H \times 4 = B8C0\ 0000\ H + 2E3\ H \ll 2 = B8C0\ 0B8C\ H。$

第三问【该页表项中的页框号是多少】答案：该页表项中的页框号是 195 H。

此处考查页表项的内容。

该页表项存储的是页表的起始物理地址，根据题意，该起始地址为 6540 0000 H，

6540 0000 H = 0110010101 000000000000000000000000 B

，高 10 为页框号 01 1001 0101 B = 195 H。

46.（本题 8 分）计算机系统中的进程之间往往需要相互协作以完成一个任务。在某网络系统中，缓冲区 B 用于存放一个数据分组，对 B 的操作有现有 C1、C2 和 C3 三种操作。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个分组，C3 对 B 中的数据分组进行修改，要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。

请回答下列问题。

- (1) 假设进程 P1、P2 均需要执行 C1，实现 C1 代码是否为临界区，为什么？（2 分）
- (2) 设 B 初始为空，进程 P1 执行 C 一次，进程 P2 执行 C2 一次，请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）
- (3) 假设 B 初始不为空，进程 P1 和 P2 各执 C3 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）

参考答案：

- (1) 实现 C1 代码是临界区，因为代码 C1 执行对 B 的写操作，任何时候只有一个进程可以执行对 B 的写操作，进程 P1、P2 均需要执行 C1，两者必须互斥执行。
- (2) 因为 B 初始为空，所以必须是进程 P1 先执行写入操作，进程 P2 后执行读取操作，两者是同步关系，同步默认信号量初始为 0，先 V(signal) 后 P(wait)，该同步伪代码如下。

semaphore syn = 0; // 用于实现进程 P1 和 P2 的同步

```
P1 {  
    ...  
    C1;  
    signal(syn);  
    ...  
}
```

```
P2 {  
    ...  
    wait(syn);  
    C2;  
    ...  
}
```

(3) 因为 B 初始不为空, 表示 B 中有数据, 可以执行 C3 的修改操作, 修改操作为互斥操作, 进程 P1 和 P2 各执 C3 一次, 两者必须互质执行, 互斥操作信号量初始为 1, 先 P(wait)后 V(signal), 中间夹互斥操作。进程 P1 和 P2 的互斥伪代码如下。

semaphore mutex = 1; // 用于实现进程 P1 和 P2 的互斥

```
P1 {  
    ...  
    wait(mutex);  
    C3;  
    signal(mutex);  
    ...  
}
```

```
P2 {  
    ...  
    wait(mutex);  
    C3;  
    signal(mutex);  
    ...  
}
```